

Module: 7

Programmable Logic Devices (PLDs)

Dr. Sukanta Kumar Tulo
SENSE, VIT Vellore

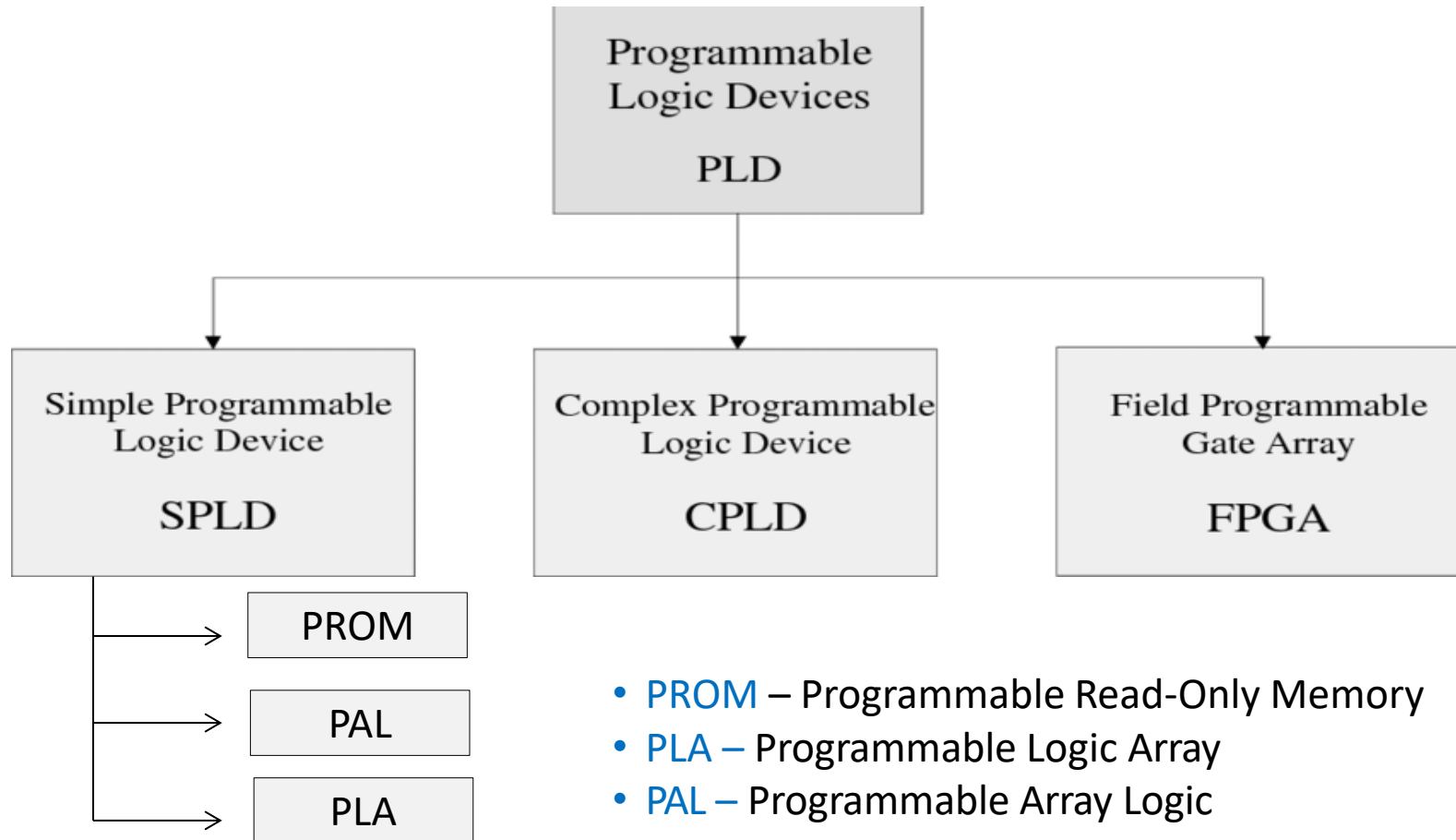
Types of Programmable Logic Devices:

PLA, PAL, CPLD, FPGA Generic Architecture.

Programmable Logic Devices

- Programmable Logic Device (PLD) is one kind of chip used to implement the logic circuit.
- It includes a set of logic circuit elements that can be modified in several ways.
- A PLD is looked like a black box that consists of programmable switches as well as logic gates.
- Can be erased electrically and reprogrammed with a new design, making them very well suited for academic and prototyping.

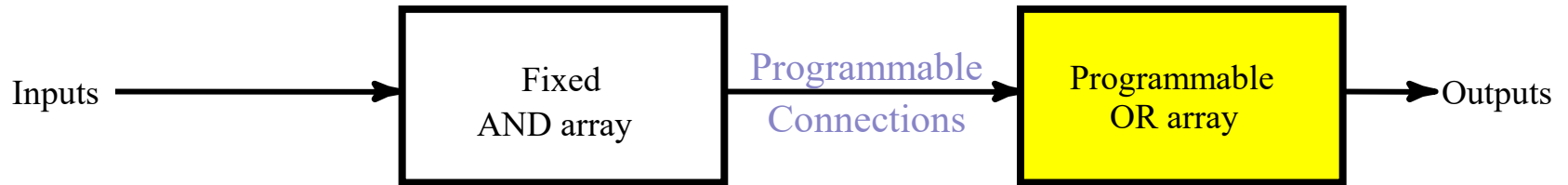
Types of Programmable Logic Devices



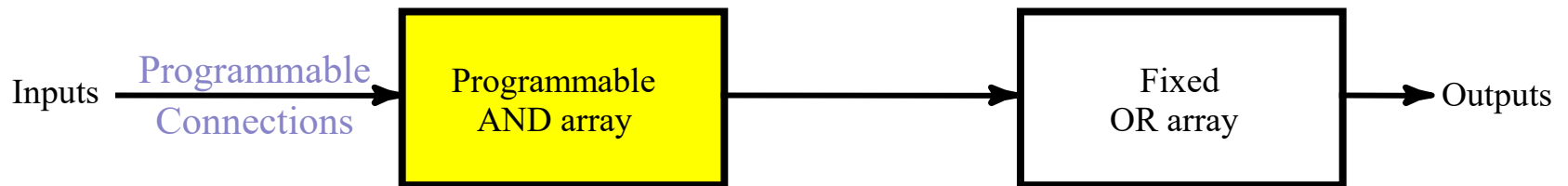
SPLD

- PROM: input connection matrix is hardwired. The user can modify the output connection matrix.
- PAL: output connection matrix is hardwired. The user can modify the input connection matrix.
- PLA: user can modify both the input connection matrix and the output connection matrix.

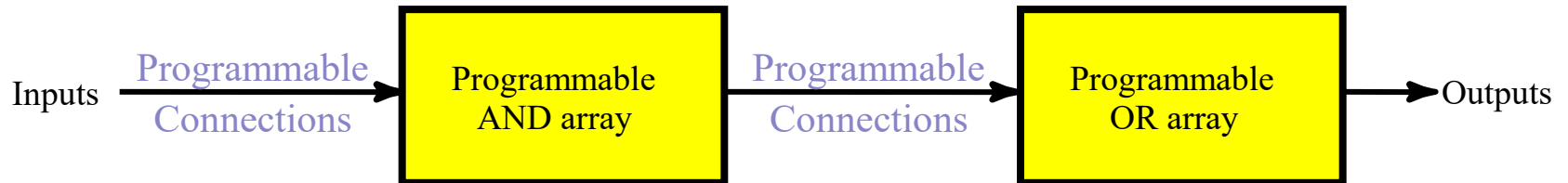
Device	AND-array	OR-array
PROM	Fixed	Programmable
PLA	Programmable	Programmable
PAL	Programmable	Fixed



(a) Programmable read-only memory (PROM)



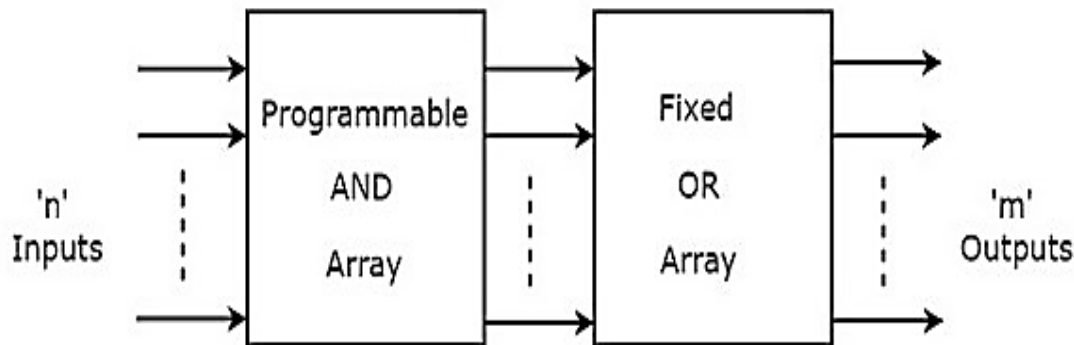
(b) Programmable array logic (PAL) device



(c) Programmable logic array (PLA) device

Programmable Array Logic (PAL)

- The programmable array logic (PAL) is a logic device with a programmable AND array and a fixed OR array.
- It is user friendly
- PAL: easier to program but not as flexible as PLA.

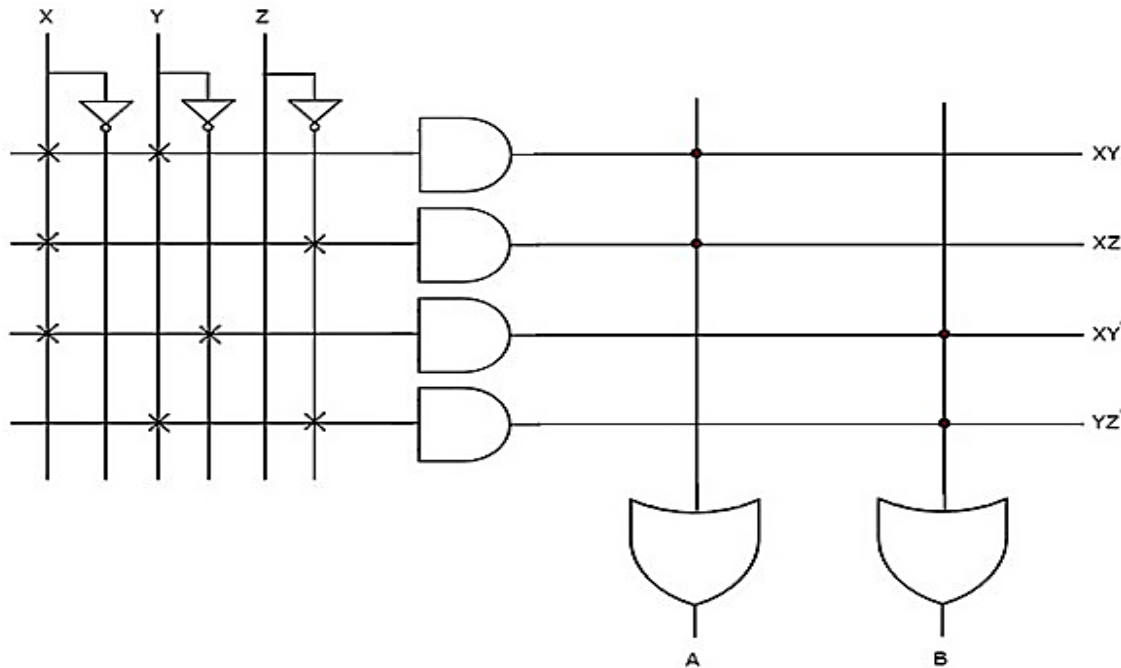


PAL

Let us implement the following **Boolean functions** using PAL.

$$A = XY + XZ'$$

$$B = XY' + YZ'$$

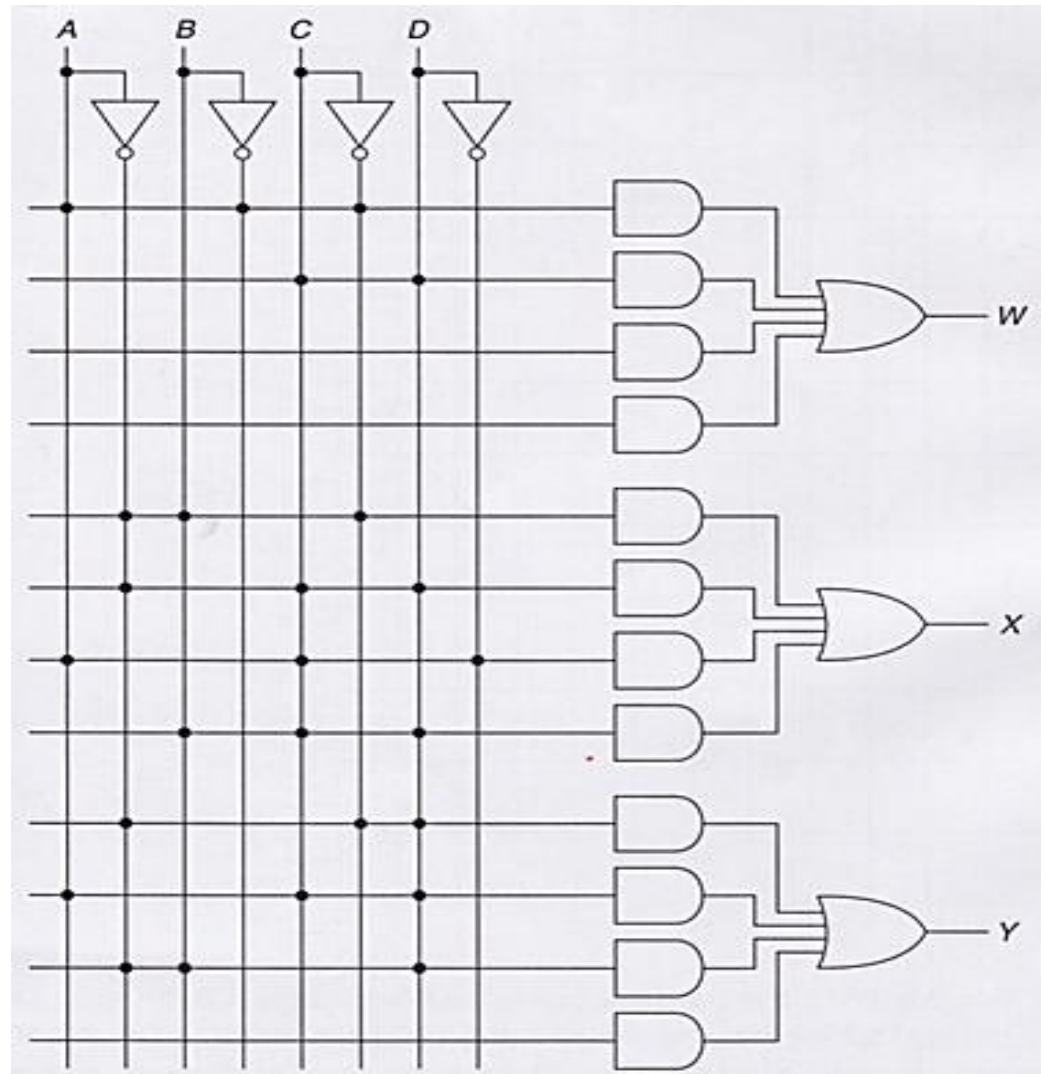


PAL

$$W = AB'C' + CD$$

$$X = A'BC' + A'CD + ACD' + BCD$$

$$Y = A'C'D + ACD + A'BD$$



Implement PAL for the given expression using

$$w(A, B, C, D) = \sum(2, 12, 13)$$

$$x(A, B, C, D) = \sum(7, 8, 9, 10, 11, 12, 13, 14, 15)$$

$$y(A, B, C, D) = \sum(0, 2, 3, 4, 5, 6, 7, 8, 10, 11, 15)$$

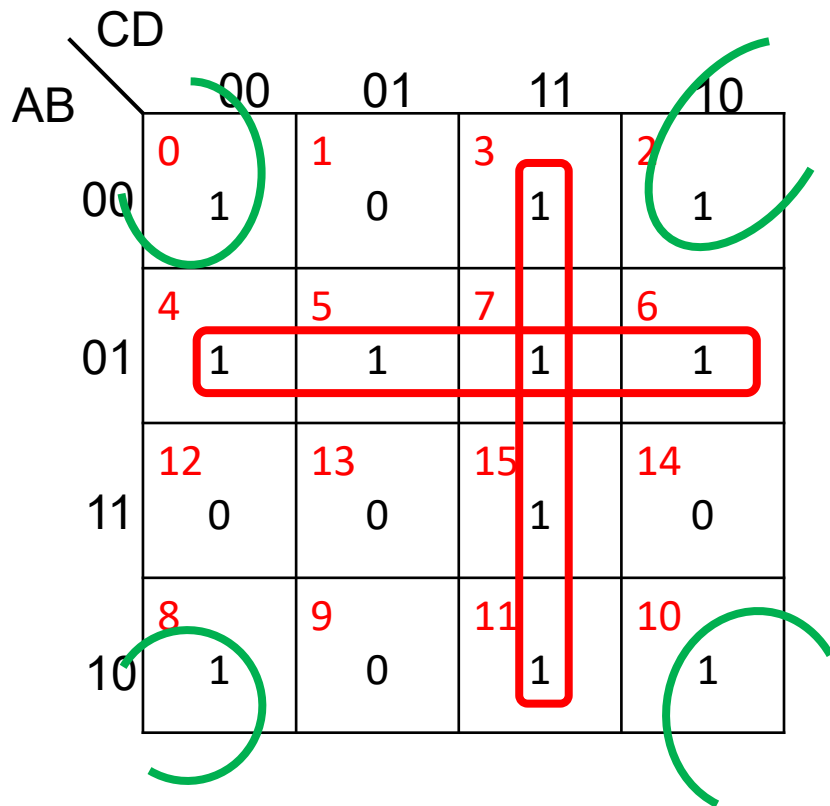
$$z(A, B, C, D) = \sum(1, 2, 8, 12, 13)$$

		CD			
		00	01	11	10
AB	00	0 0	1 0	3 0	2 1
	01	4 0	5 0	7 0	6 0
	11	12 1	13 1	15 0	14 0
	10	8 0	9 0	11 0	10 0

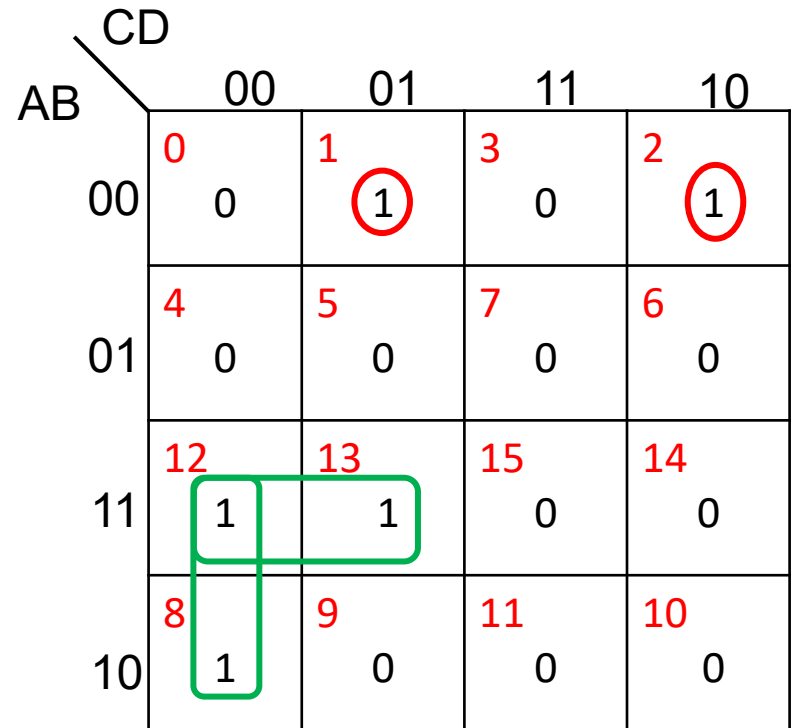
$$w = AB\bar{C} + \bar{A}\bar{B}C\bar{D}$$

		CD			
		00	01	11	10
AB	00	0 0	1 1	3 0	2 1
	01	4 0	5 0	7 1	6 0
	11	12 1	13 1	15 1	14 1
	10	8 1	9 1	11 1	10 1

$$x = A + BCD$$



$$Y = \bar{A}B + CD + \bar{B}\bar{D}$$

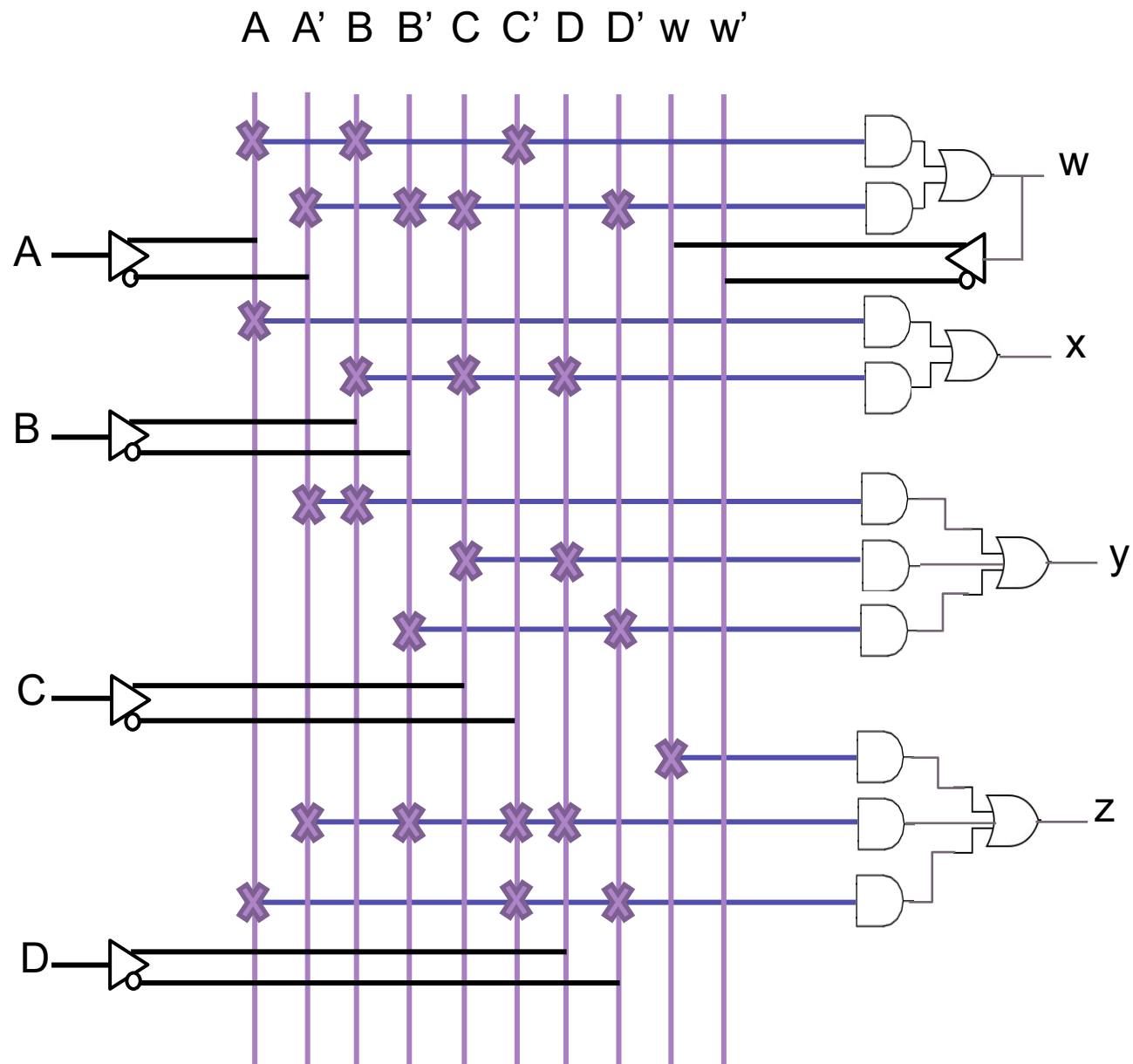


$$z = AB\bar{C} + \bar{A}\bar{B}C\bar{D} + \bar{A}\bar{B}\bar{C}D + A\bar{C}\bar{D}$$

$$z = \underbrace{AB\bar{C} + \bar{A}\bar{B}C\bar{D} + \bar{A}\bar{B}\bar{C}D}_w + A\bar{C}\bar{D}$$

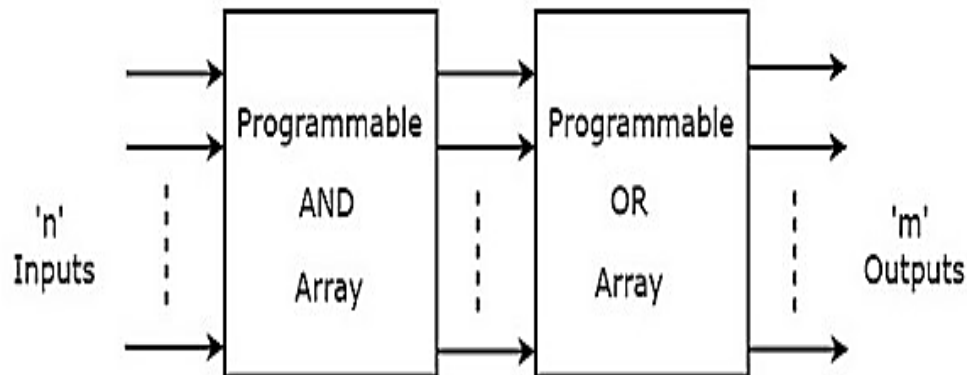
PAL Program Table

Product Term	AND Inputs					Outputs
	A	B	C	D	W	
1	1	1	0	-	-	W = ABC' + A'B'CD'
2	0	0	1	0	-	
3	-	-	-	-	-	
4	1	-	-	-	-	X = A + BCD
5	-	1	1	1	-	
6	-	-	-	-	-	
7	0	1	-	-	-	Y = A'B + CD + B'D'
8	-	-	1	1	-	
9	-	0	-	0	-	
10	-	-	-	-	1	Z = w + A'B'C'D + AC'D'
11	0	0	0	1	-	
12	1	-	0	0	-	

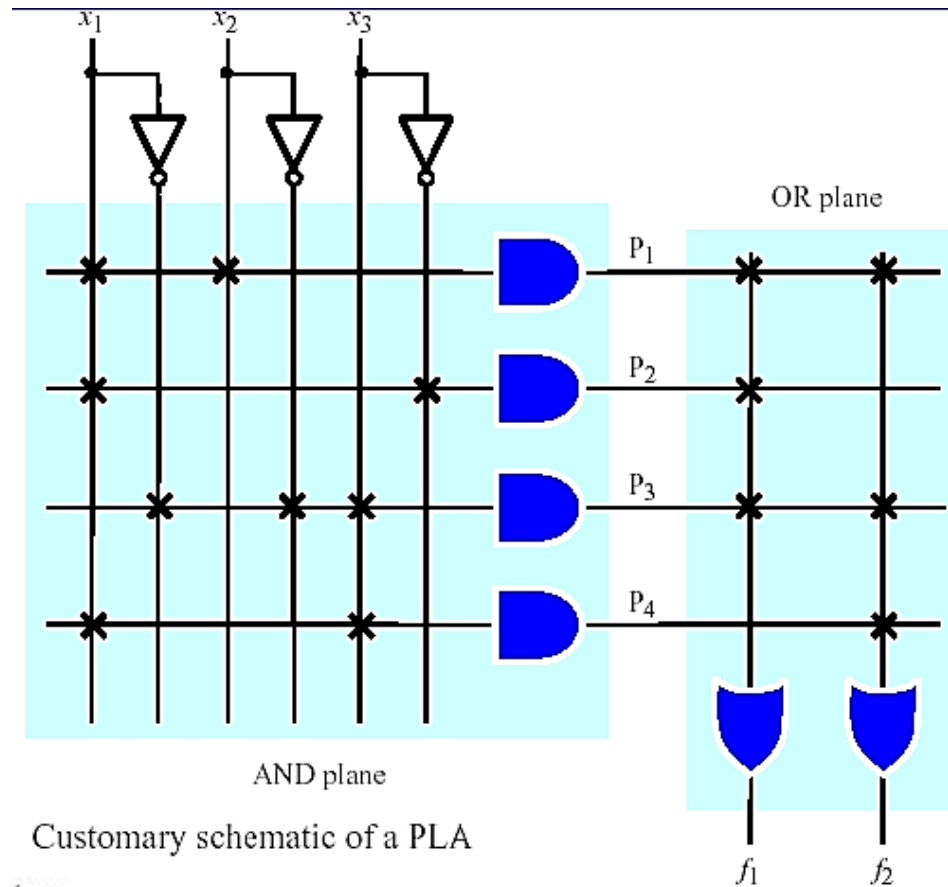


Programmable Logic Array (PLA)

- PLA is a programmable logic device that has both Programmable AND array & Programmable OR array.
- It is the most flexible PLD.



PLA

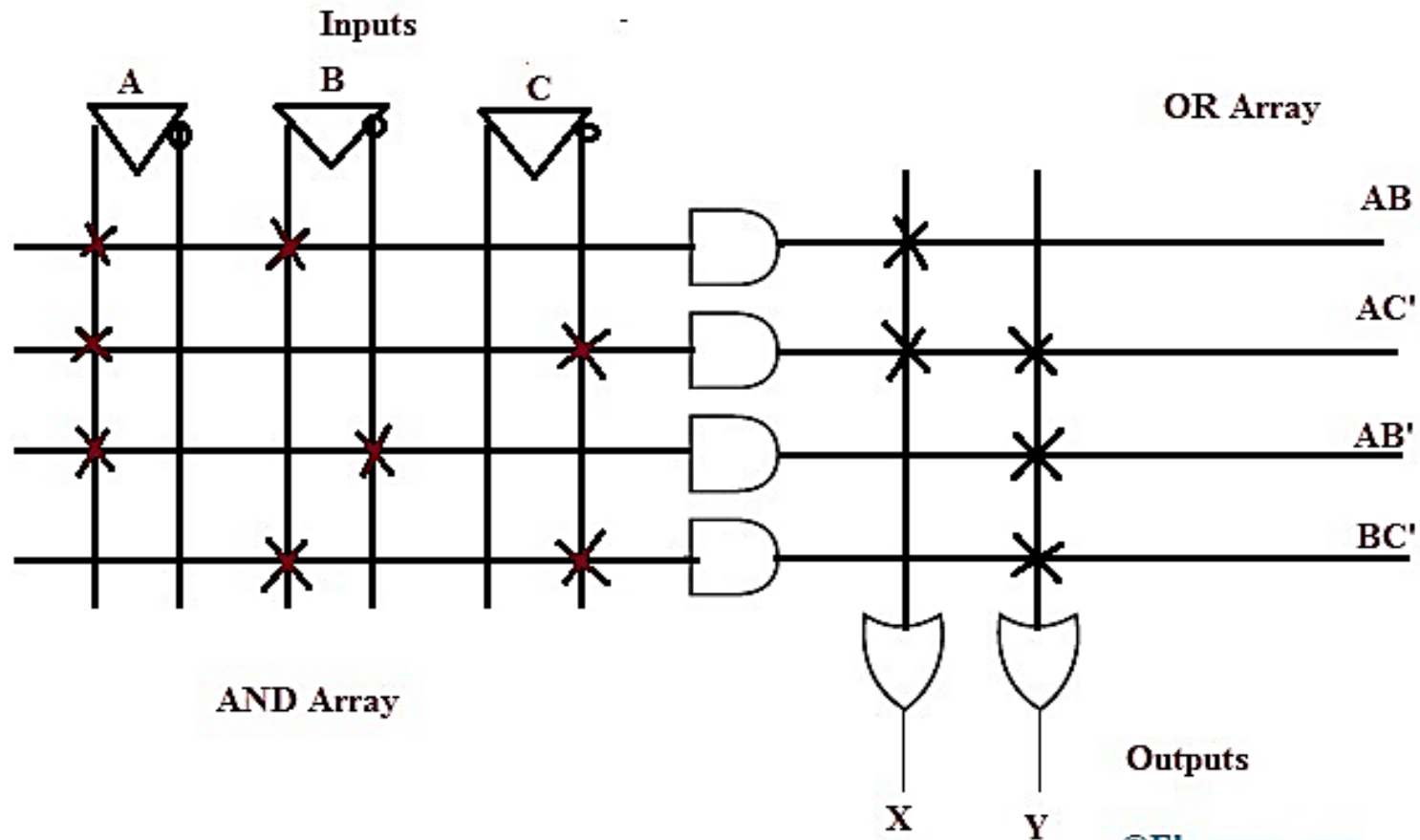


- A 3×2 PLA with 4 product terms.

PLA

$$X = AB + AC'$$

$$Y = AB' + BC + AC'$$

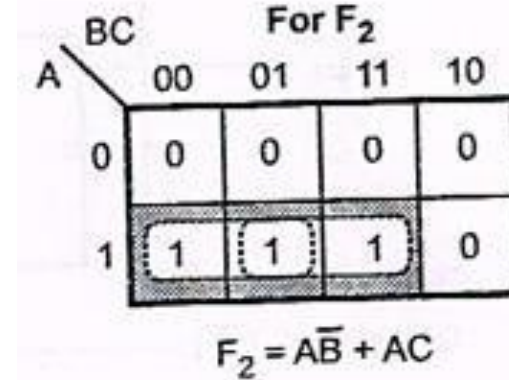
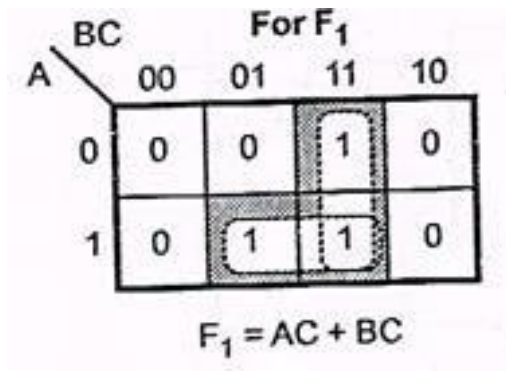


- **Example:** A combinational circuit is defined by the functions :
- $F_1 = \sum m(3,5,7)$
- $F_2 = \sum m(4,5,7)$
- Implement the circuit with a PLA having 3 inputs, 3 product terms and two outputs.
- **Solution :** Let us determine the truth table for the given Boolean functions

Table (1): Truth table

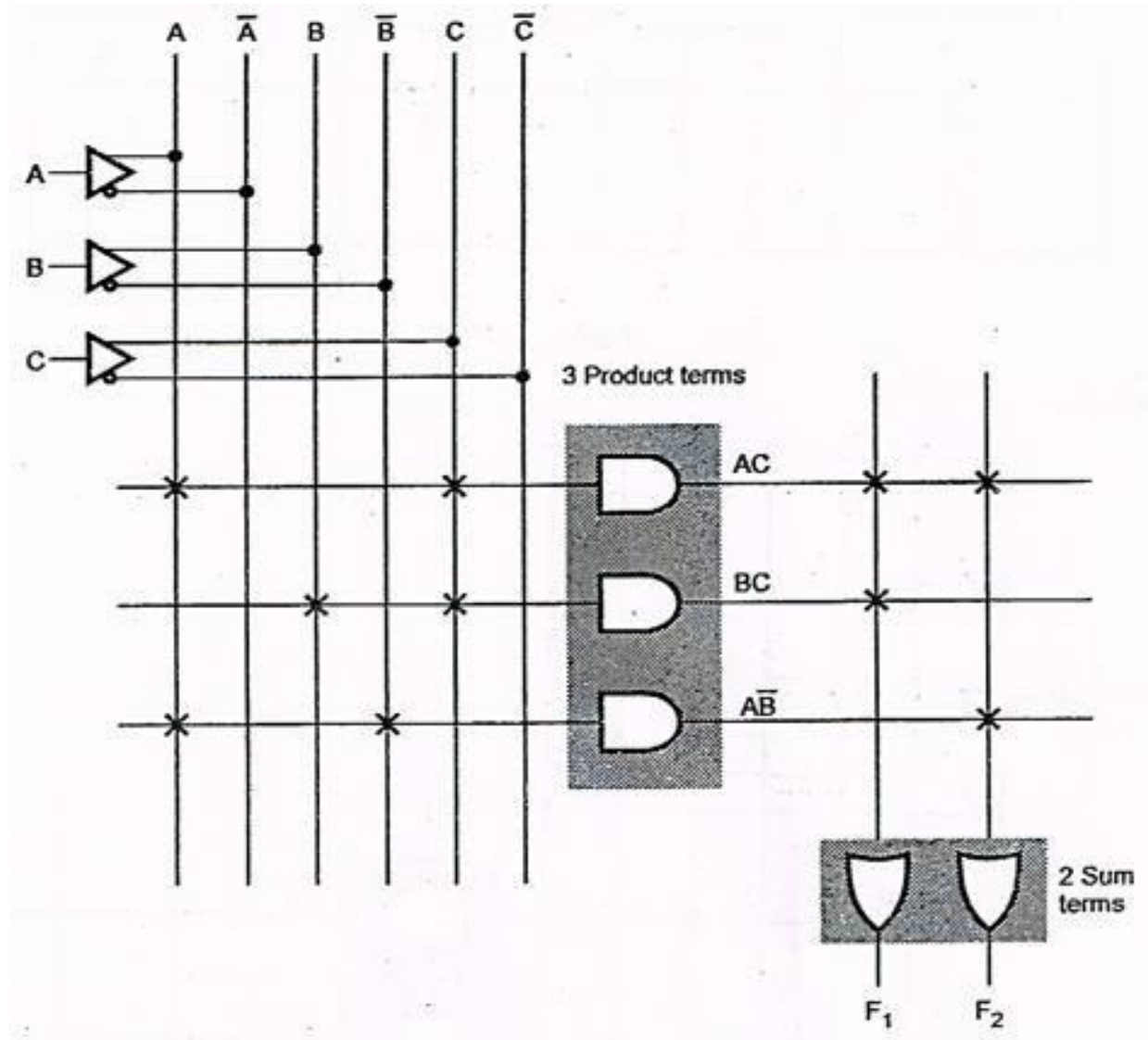
A	B	C	F ₁	F ₂
0	0	0	0	0
0	0	1	0	0
0	1	0	0	0
0	1	1	1	0
1	0	0	0	1
1	0	1	1	1
1	1	0	0	0
1	1	1	1	1

K-map simplification



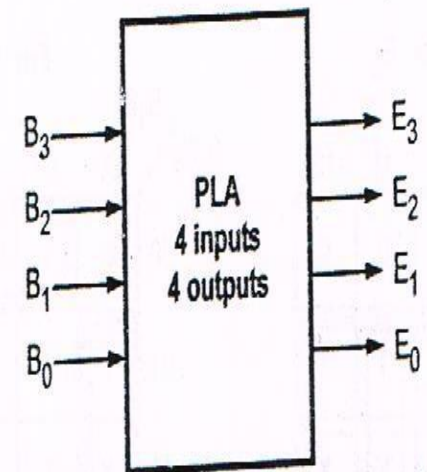
PLA program table

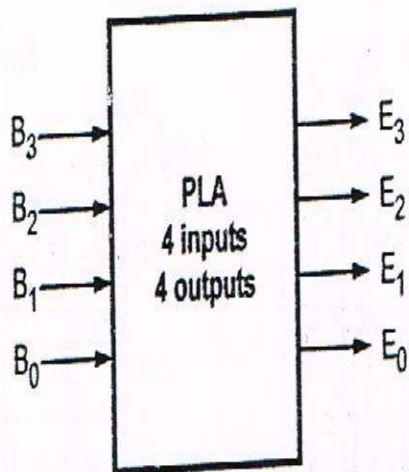
	Product term	Inputs			Outputs	
		A	B	C	F_1	F_2
AC	1	1	-	1	1	1
BC	2	-	1	1	1	-
$A\bar{B}$	3	1	0	-	-	1



Example: Design a BCD to Excess-3 code converter and implement using suitable PLA.

Decimal	BCD code				Excess-3 code			
	B ₃	B ₂	B ₁	B ₀	E ₃	E ₂	E ₁	E ₀
0	0	0	0	0	0	0	1	1
1	0	0	0	1	0	1	0	0
2	0	0	1	0	0	1	0	1
3	0	0	1	1	0	1	1	0
4	0	1	0	0	0	1	1	1
5	0	1	0	1	1	0	0	0
6	0	1	1	0	1	0	0	1
7	0	1	1	1	1	0	1	0
8	1	0	0	0	1	0	1	1
9	1	0	0	1	1	1	0	0





For E_3

$B_3B_2 \backslash B_1B_0$	00	01	11	10
00	0	0	0	0
01	0	1	1	1
11	X	X	X	X
10	1	1	X	X

$$E_3 = B_3 + B_2\bar{B}_0 + B_2B_1$$

For E_2

$B_3B_2 \backslash B_1B_0$	00	01	11	10
00	0	1	1	1
01	1	0	0	0
11	X	X	X	X
10	0	1	X	X

$$E_2 = B_2\bar{B}_1\bar{B}_0 + \bar{B}_2B_0 + \bar{B}_2B_1$$

For E_1

$B_3B_2 \backslash B_1B_0$	00	01	11	10
00	1	0	1	0
01	1	0	1	0
11	X	X	X	X
10	1	0	X	X

$$E_1 = \bar{B}_1\bar{B}_0 + B_1B_0$$

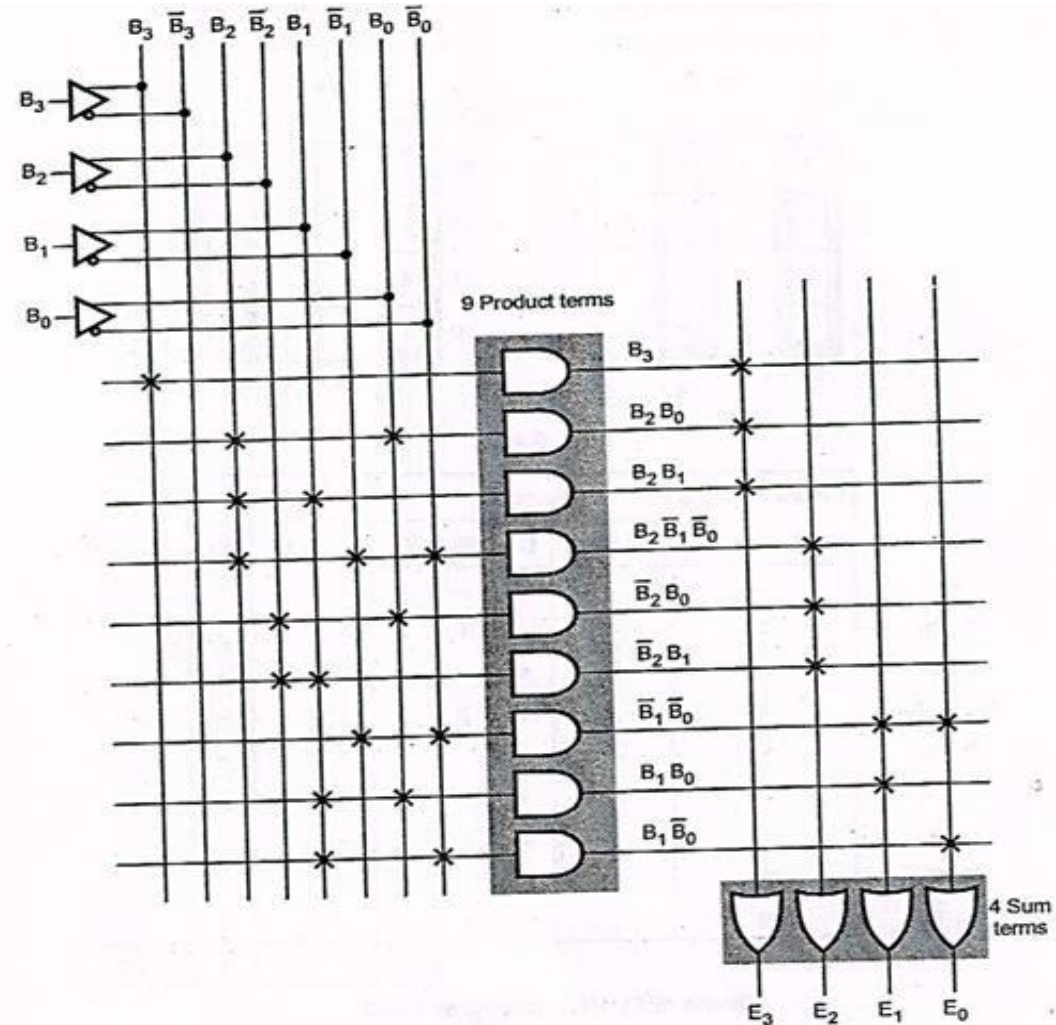
For E_0

$B_3B_2 \backslash B_1B_0$	00	01	11	10
00	1	0	0	1
01	1	0	0	1
11	X	X	X	X
10	1	0	X	X

$$E_0 = \bar{B}_1\bar{B}_0 + B_1\bar{B}_0$$

PLA program table

	Product terms	Inputs				Outputs			
		B_3	B_2	B_1	B_0	E_3	E_2	E_1	E_0
B_3	1	1	-	-	-	1	-	-	-
$B_2 B_0$	2	-	1	-	1	1	-	-	-
$B_2 B_1$	3	-	1	1	-	1	-	-	-
$B_2 \bar{B}_1 \bar{B}_0$	4	-	1	0	0	-	1	-	-
$\bar{B}_2 B_0$	5	-	0	-	1	-	1	-	-
$\bar{B}_2 B_1$	6	-	0	1	-	-	1	-	-
$\bar{B}_1 \bar{B}_0$	7	-	-	0	0	-	-	1	1
$B_1 B_0$	8	-	-	1	1	-	-	1	-
$B_1 \bar{B}_0$	9	-	-	1	0	-	-	-	1



Implement the following Boolean function with a PLA?

$$F1(A,B,C) = \sum(0,1,2,4)$$

$$F2(A,B,C) = \sum(0,5,6,7)$$

$$F3(A,B,C) = \sum (0,3,5,7)$$

Solution:

K – Map Simplification

For F_1

A \ BC	00	01	11	10
0	1	1	0	1
1	1	0	0	0

$$F_1 = BC + AB + AC$$

For F_2

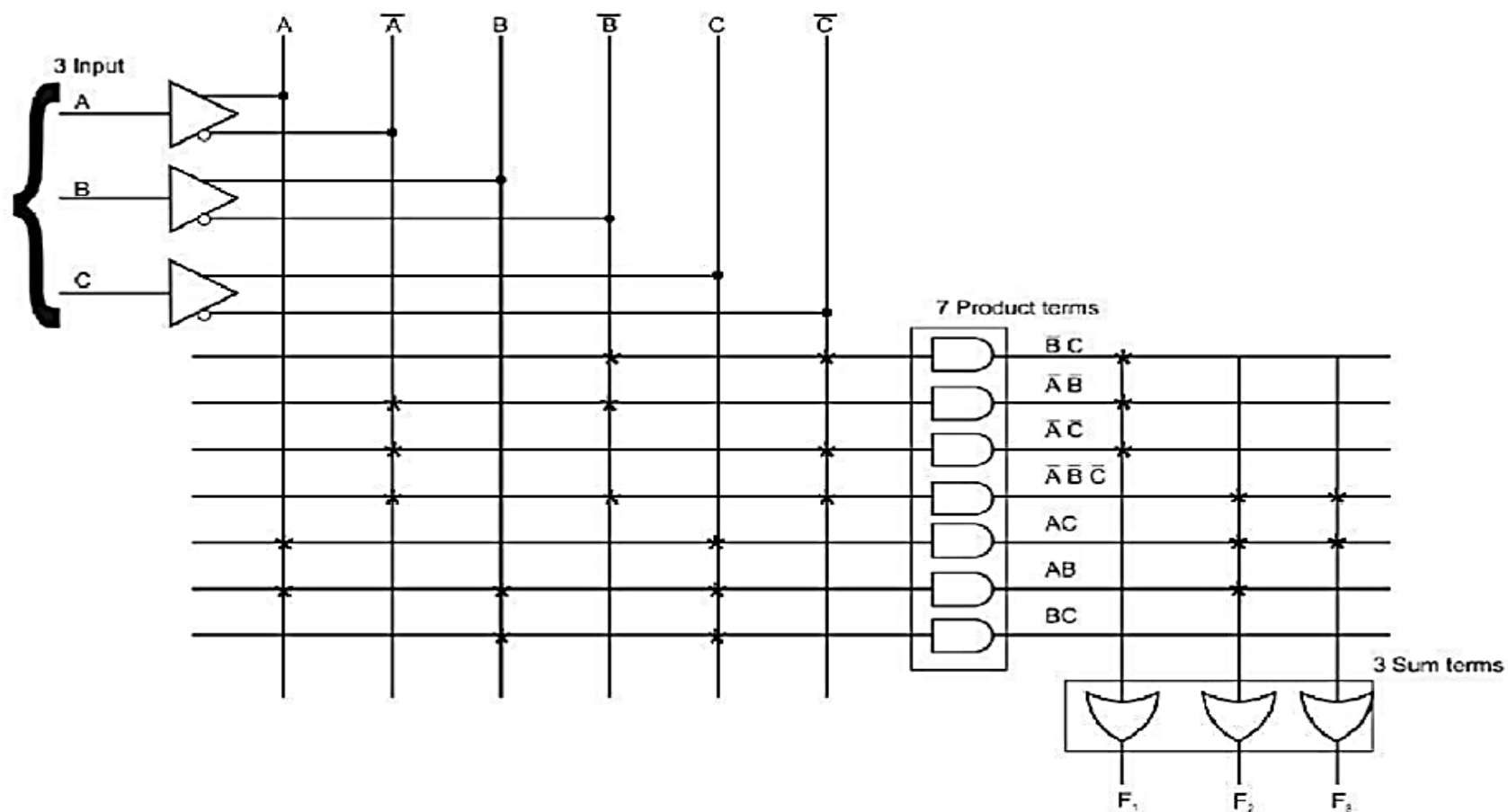
A \ BC	00	01	11	10
0	1	0	0	0
1	0	1	1	1

$$F_2 = ABC + AC + AB$$

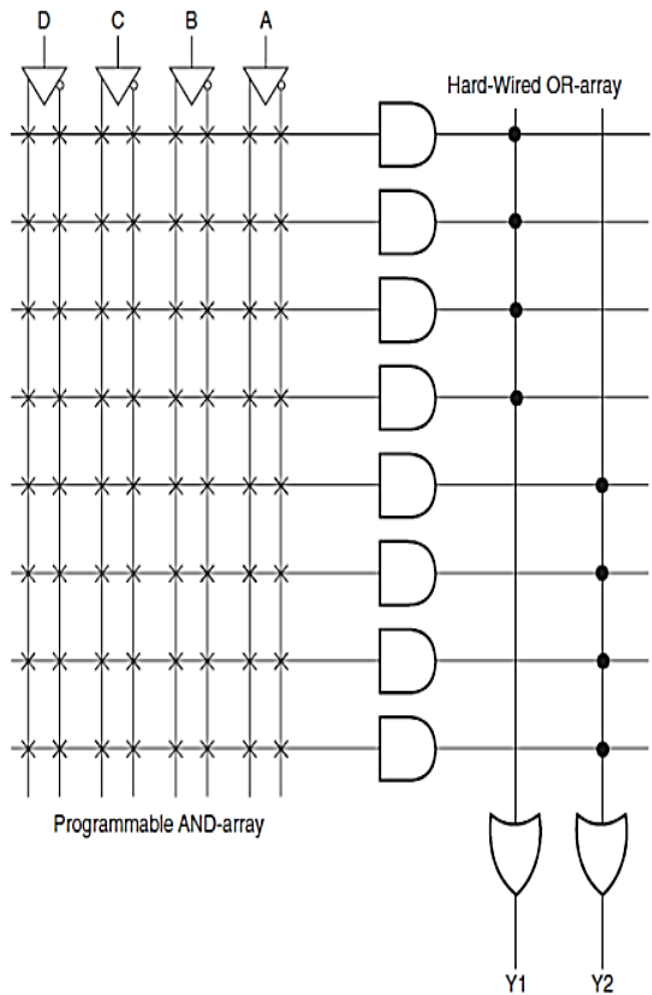
For F_3

A \ BC	00	01	11	10
0	1	0	1	0
1	0	1	1	0

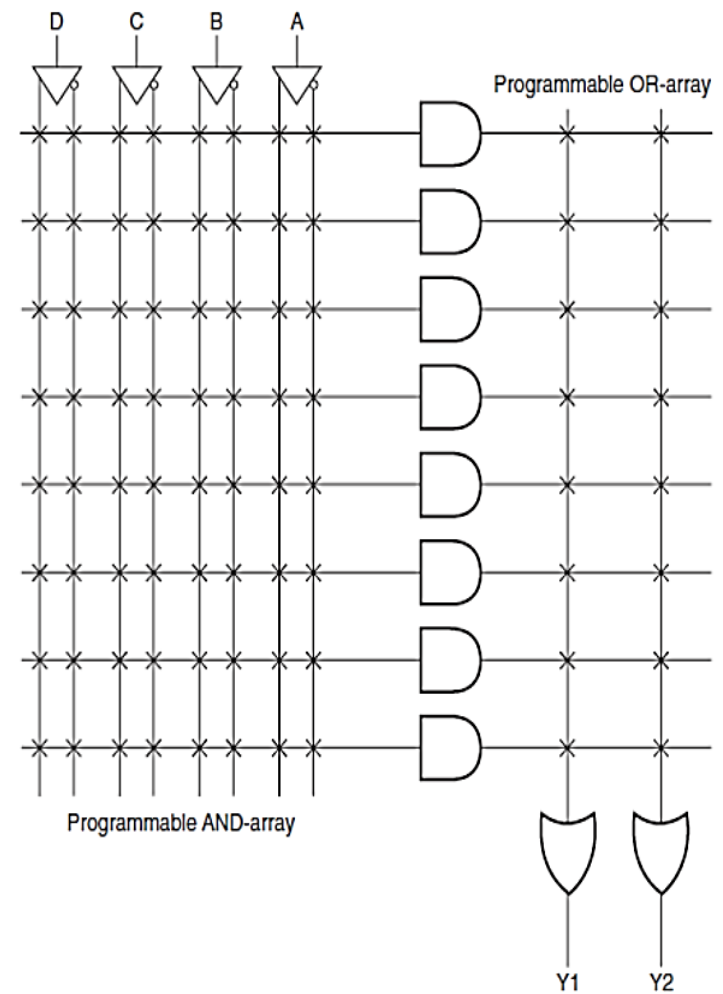
$$F_3 = \bar{A}BC + BC + AC$$



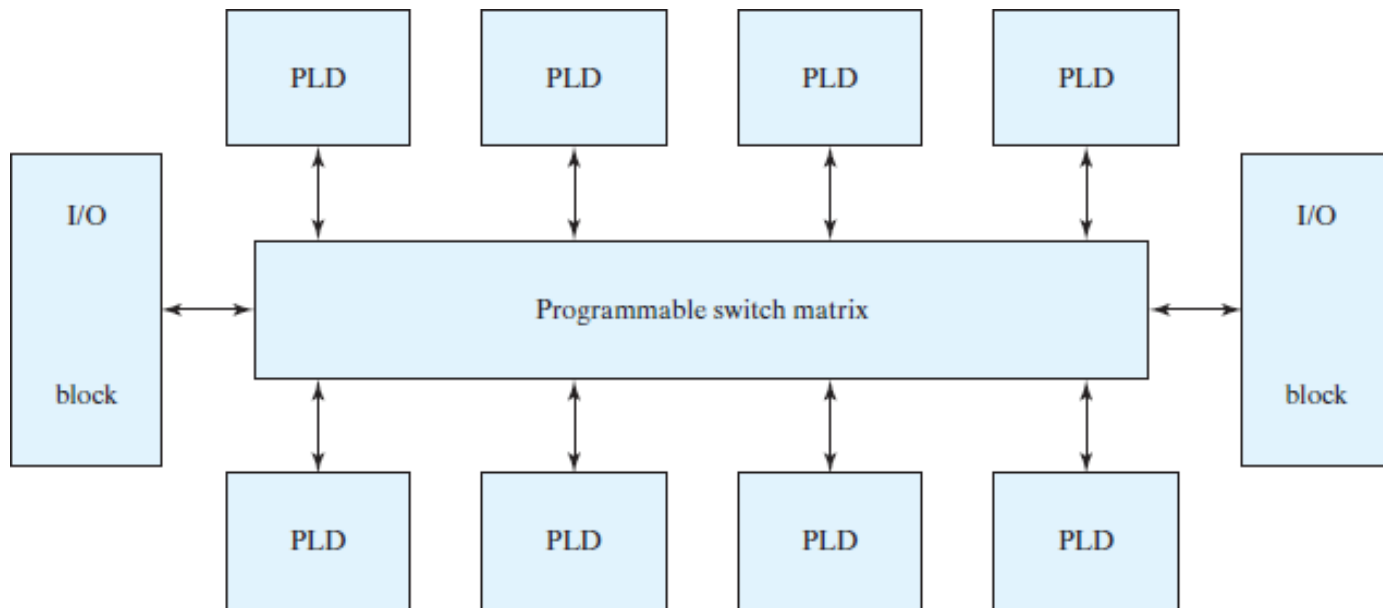
Programmable Array Logic



Programmable Logic Array



Complex Programmable Logic Devices (CPLD)



CPLD

- The CPLD consists of multiple PLDs interconnected through a programmable switch matrix.
- The input – output blocks provide the connections to the IC pins.
- Each I/O pin is driven by a three state buffer and can be programmed to act as inputs or outputs.
- The switch matrix receives inputs from the I/O block and directs them to the individual macrocells.
- Similarly selected outputs from macrocells are sent to the outputs as needed.
- Each PLD contains 8 to 16 macrocells, usually fully connected.
- If a macrocell has unused product terms, they can be used by other nearby macrocells.

Need of CPLD

- The Programmable Logic Devices (PLDs) such as PLAs and PALs have **limited number of inputs, product terms and outputs**. These devices can **support up to about 32** total number of inputs and outputs only.
- For implementation of circuits that require **more inputs and outputs** than that are available in a single SPLD chip, either **multiple SPLD chips** can be employed. But this has also some limitations.
- To overcome these limitations, more sophisticated chip like **CPLD is needed** to design the digital circuits.

Concept of CPLD

- The complexity of any digital IC chip can be specified in terms of number of equivalent 2-input NAND gates.
- A typical PAL has 8 macro cells, if each macro cell represents about 20 equivalent gates, then the PAL can accommodate a circuit that needs up to about 160 gates.
- For a circuit requiring very large number of gates, CPLDs having large number of macrocells (say 512 macro cells) can implement circuits of up to about 10 thousand equivalent gates i.e. they are similar to SPLDs except that the CPLD is equivalent of 2 to 64 SPLDs.
- A CPLD typically contains from tens to a few hundred macrocells.
- They are digital ICs that are just like a large number of PALs in a single silicon chip connected to each other through a cross point switch.

Architecture of CPLD

- It consists of a **number of PAL like blocks**, **input/output blocks** and a set of **interconnection wires**.
- The **PAL** like blocks are connected to a set of **interconnection wires** and to an **input/output block**.
- The **input block** is used to drive signals to the pins of the CPLD at the appropriate voltage levels with the appropriate current.
- A **PAL-like block (also called functional block)** usually consists of **16 macro cells**.
- Each **macro cell** consists of an **AND-OR configuration**, an **EX-OR gate** a **flip-flop**, a **multiplexer** and a **tristate buffer**.
- Each **AND-OR configuration** usually consists of 5 to 20 AND gates and an OR gate with 5 to 20 inputs.
- An **EX-OR gate** is used to obtain the output of **OR gate in an inverted or non-inverted** form depending upon its other input being 1 or 0 respectively.
- A **tristate buffer** acts as a switch, which enables the chips pin to be used output or as an input.
- In case of chip pin is used as an input pin an external source can drive a signal on to the pin, which can be connected to other macro cells using the interconnection wiring.
- When used as an input pin, the macro cell becomes redundant and it is wasted.

Advantages of CPLDs

- **Ease of Design:** With use of HDL and CPLD development tools.
- **Reduced Board Area.**
- Available in **tiny sizes.**
- Due to **less maintenance, lower Development Costs**

Field-Programmable Gate Arrays

Why FPGA:

PAL or PLA : it uses Macrocell represents about 20 equivalent gates. A PAL that has eight Macrocells can accommodate a circuit that needs up to about 160 gates

SPLDs and CPLDs : a large CPLD that has 500 Macrocells can implement circuits of up to about 10,000 equivalent gates.

- ✓ By modern standards, a logic circuit with **10,000 gates is not large**. To implement larger circuits, it is convenient to use a different type of chip that has a larger logic capacity.
- ✓ A **field-programmable gate array (FPGA)** is a programmable logic device that supports implementation of relatively large logic circuits.
- ✓ FPGAs are quite different from SPLDs and CPLDs because FPGAs do not contain AND or OR planes.

Field Programmable Gate Array

- FPGA is a VLSI circuit that can be programmed at the user's location.
- A typical FPGA consists of an array of millions of logic blocks, surrounded by programmable input and output blocks and connected together via programmable interconnections.
- Its function is defined by a user's program rather than by the manufacturer of the device.
- Depending on the particular device, the program is either 'burned' in permanently or semi-permanently as part of a board assembly process, or is loaded from an external memory each time the device is powered up.
- The Field-Programmable Gate Arrays provide the benefits of custom CMOS VLSI, while avoiding the initial cost and time delay.

Field Programmable Gate Array

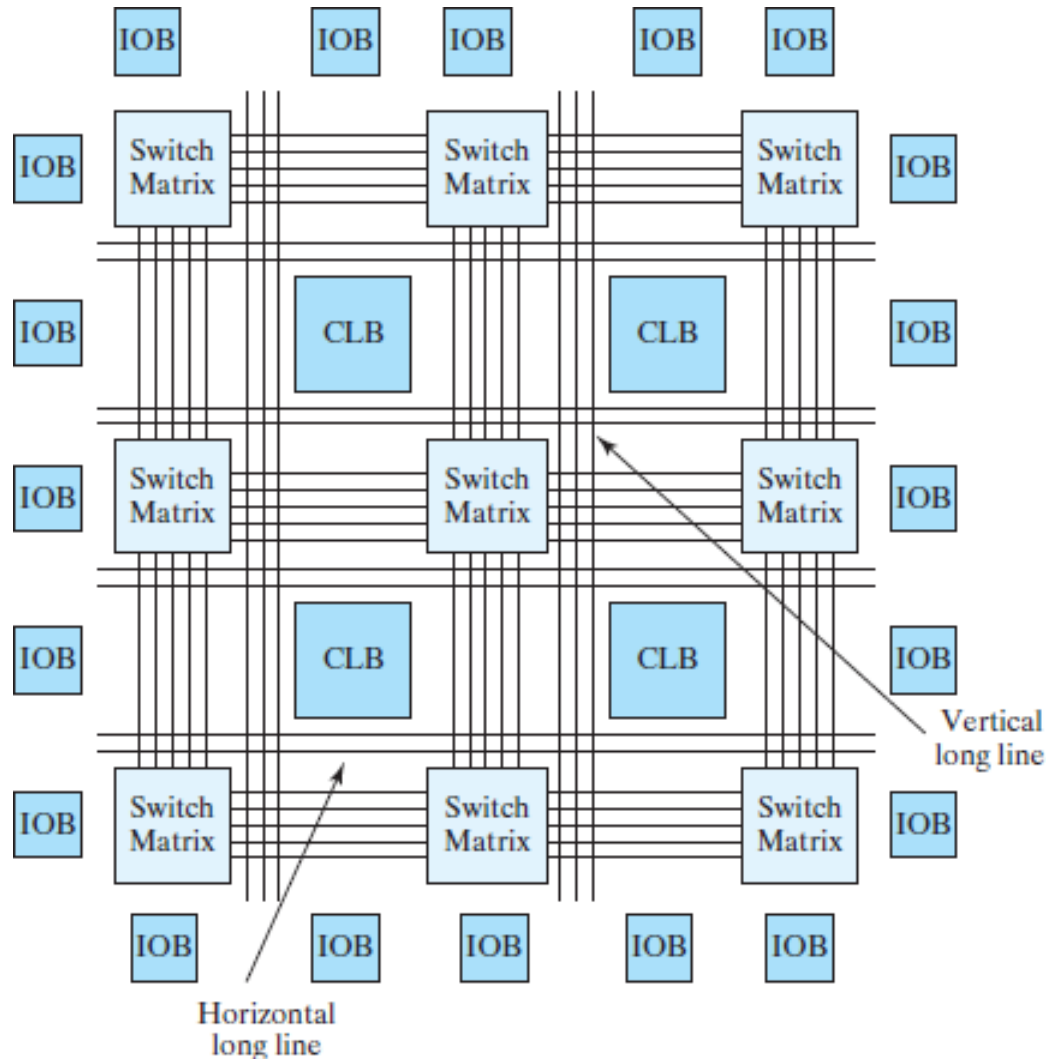
- A typical FPGA logic block consists of lookup tables, multiplexers, gates and flip flops.
- A lookup table is a truth table stored in an SRAM and provides the combinational circuit functions for the logic block.
- The design with PLD, CPLD or FPGA requires CAD tools to facilitate the synthesis procedure.
- The configuration of the FPGA architecture is generally specified using a language, i.e., HDL (Hardware Description language).

FPGA

- Before the advent of programmable logic, custom logic circuits were built at the board level using standard components, or at the gate level in expensive application-specific (custom) integrated circuits.
- FPGA is an integrated circuit that contains many (64 to over 10,000) identical logic cells that can be viewed as standard components. Each logic cell can independently take on any one of a limited set of personalities.
- Individual cells are interconnected by a matrix of wires and programmable switches. A user's design is implemented by specifying the simple logic function for each cell and selectively closing the switches in the interconnect matrix.
- Array of logic cells and interconnect form a fabric of basic building blocks for logic circuits. Complex designs are created by combining these basic blocks to create the desired circuit

Internal architecture of a typical FPGA

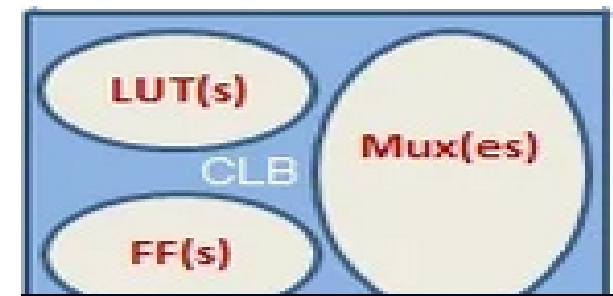
Basic architecture of Xilinx Spartan



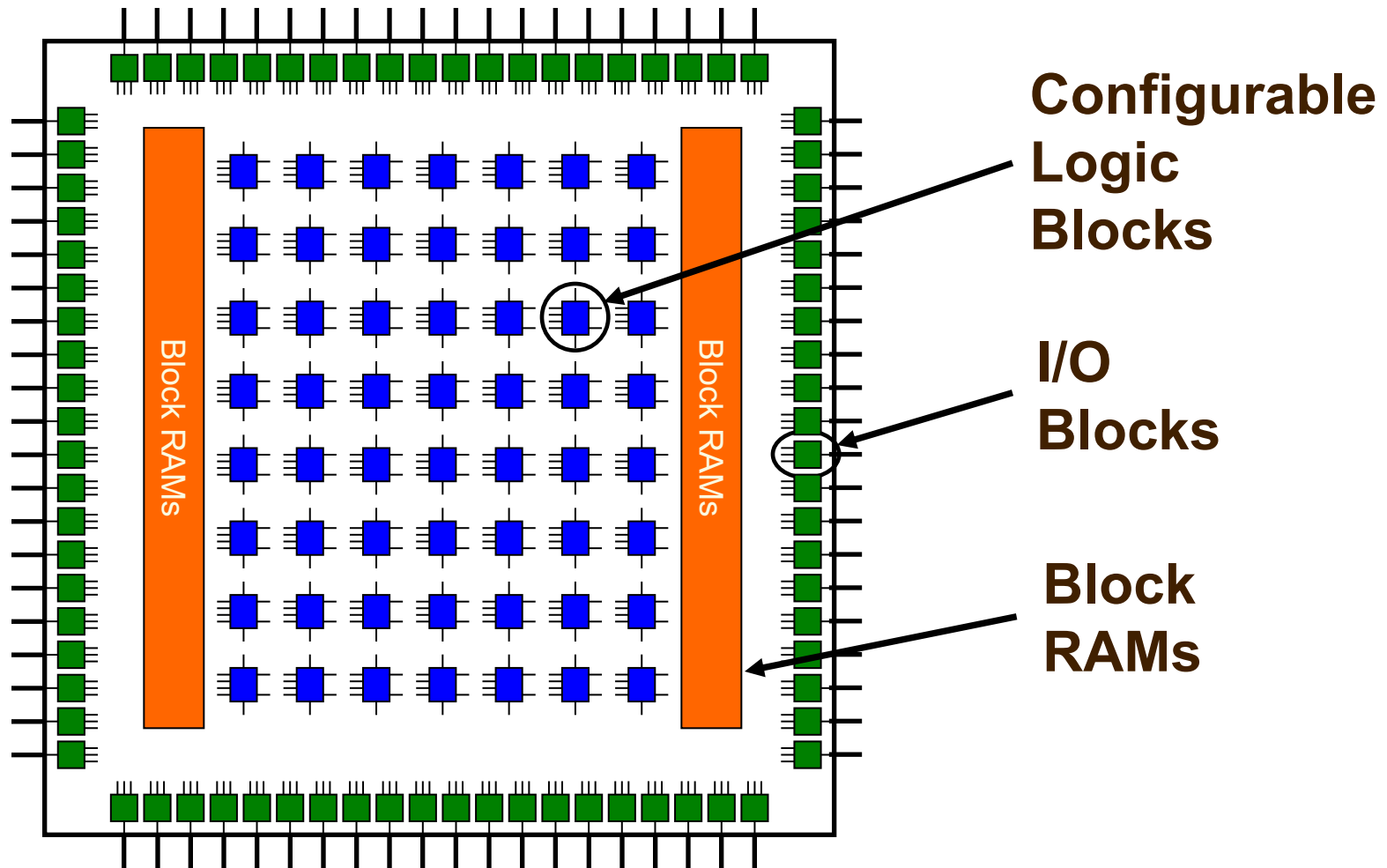
IOB (Input/Output Block)

CLB (Configurable Logic Block)

LUT (Look up table)



What is an FPGA?

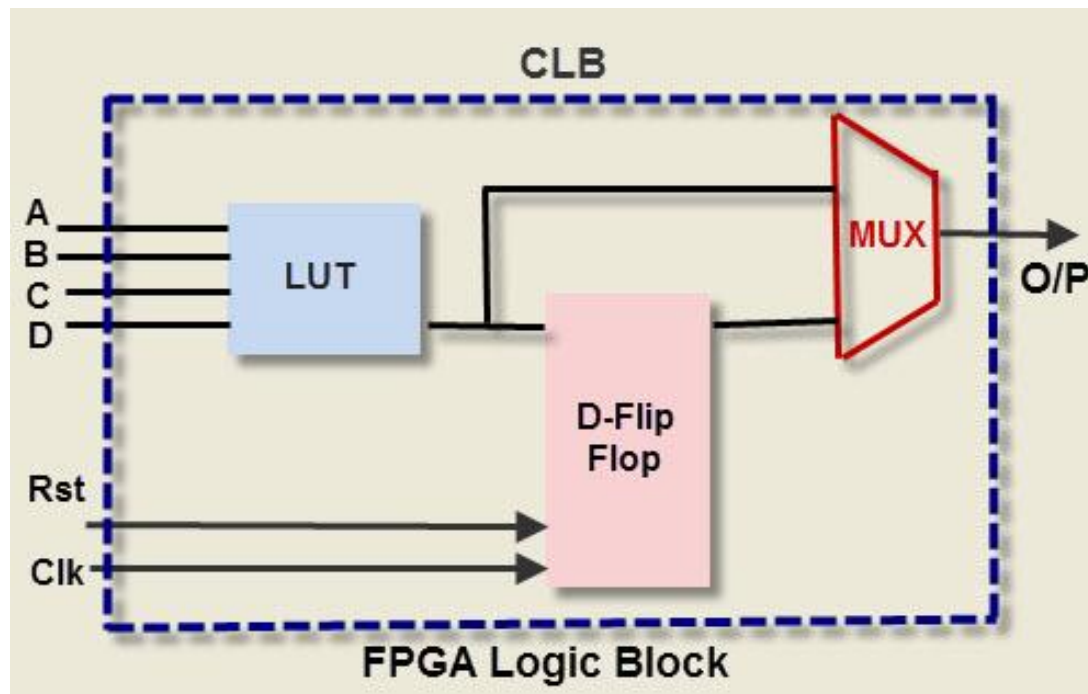


FPGA architecture

- The basic FPGA architecture has two dimensional arrays of logic blocks with a means for a user to arrange the interconnection between the logic blocks.
- The general FPGA architecture consists of four types of modules.
 1. I/O blocks or Pads
 2. Configurable logic blocks (CLB)
 3. Switch Matrix
 4. Interconnection Wires

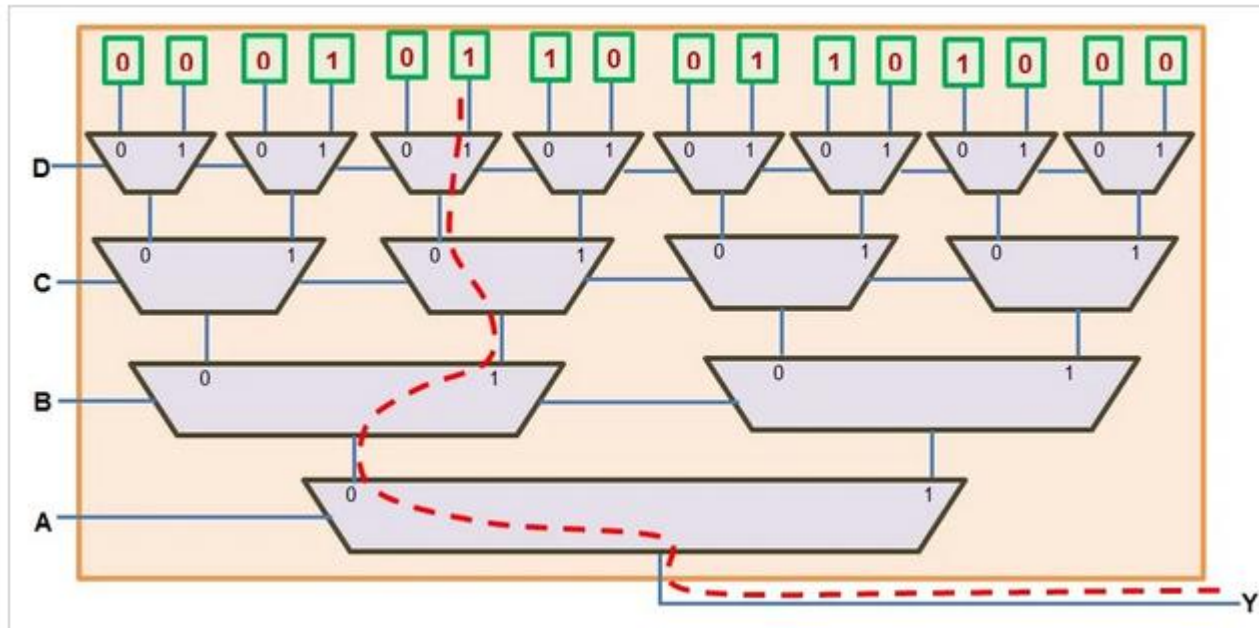
- The functions of an FPGA architecture module are discussed below:
 1. CLB (Configurable Logic Block) includes digital logic, inputs, outputs. It implements the user logic. CLB also be referred to as a logic block (LB), a logic element (LE) or a logic cell (LC).
 2. Interconnects provide direction between the logic blocks to implement the user logic.
 3. Depending on the logic, switch matrix provides switching between interconnects.
 4. I/O Pads used for the outside world to communicate with different applications.

- Logic Block contains **MUX (Multiplexer)**, **D flip flop** and **LUT**.
- LUT implements the combinational logical functions; the MUX is used for selection logic, and D flip flop stores the output of the LUT



- The basic building block of the FPGA is the Look Up Table based function generator.
- The number of inputs to the LUT vary from 3,4,6, and even 8.
- An LUT stores a predefined list of logic outputs for any combination of inputs.
- Present FPGA consists of about hundreds or thousands of configurable logic blocks.

Eg. If input ABCD = 0101



Truth Table

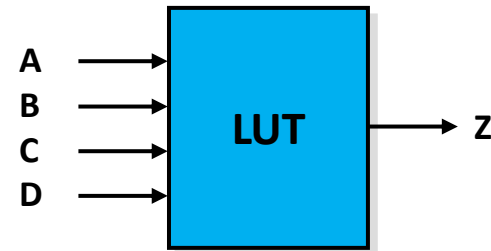
Inputs				Output
A	B	C	D	Y
0	0	0	0	0
0	0	0	1	0
0	0	1	0	0
0	0	1	1	1
0	1	0	0	0
0	1	0	1	1
0	1	1	0	1
0	1	1	1	0
1	0	0	0	0
1	0	0	1	1
1	0	1	0	1
1	0	1	1	0
1	1	0	0	1
1	1	0	1	0
1	1	1	0	0
1	1	1	1	0

Look-Up Tables (LUT)

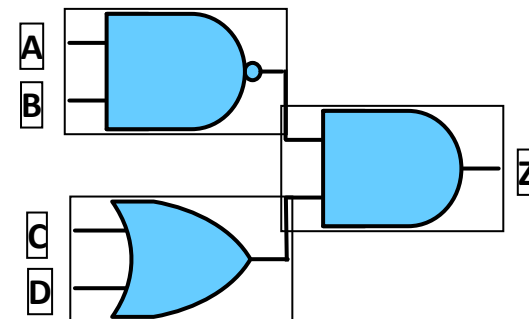
- Look-up table with N-inputs can be used to implement any combinatorial function of N inputs
- LUT is programmed with the truth-table

A	B	C	D	Z
0	0	0	0	0
0	0	0	1	1
0	0	1	0	1
0	0	1	1	1
0	1	0	0	0
0	1	0	1	1
0	1	1	0	1
0	1	1	1	1
1	0	0	0	0
1	0	0	1	1
1	0	1	0	1
1	0	1	1	1
1	1	0	0	0
1	1	0	1	0
1	1	1	0	0

Truth-table

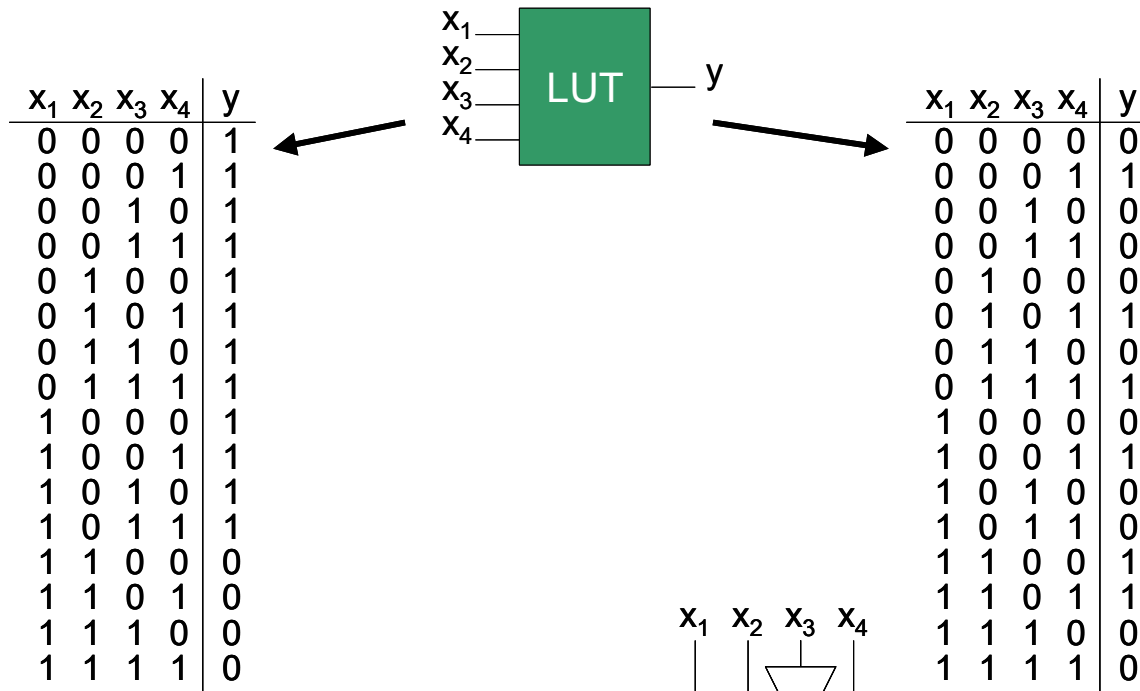


LUT implementation

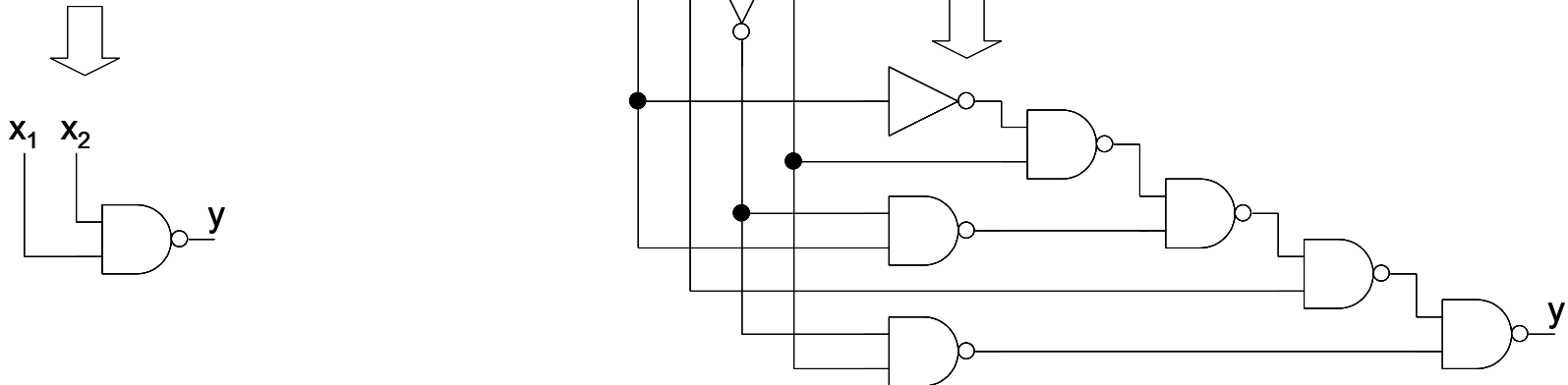


Gate implementation

LUT (Look-Up Table) Functionality

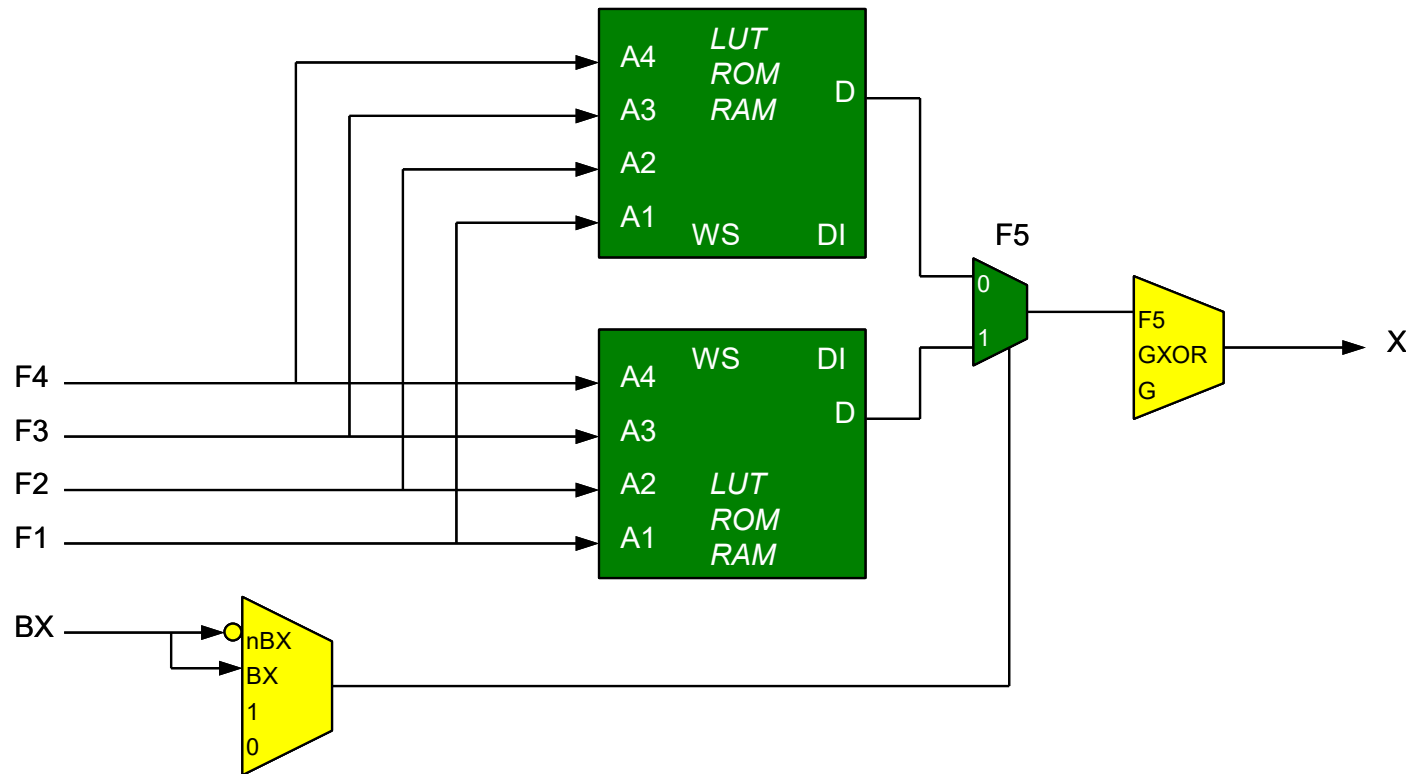


Each LUT can implement any function of 4 inputs

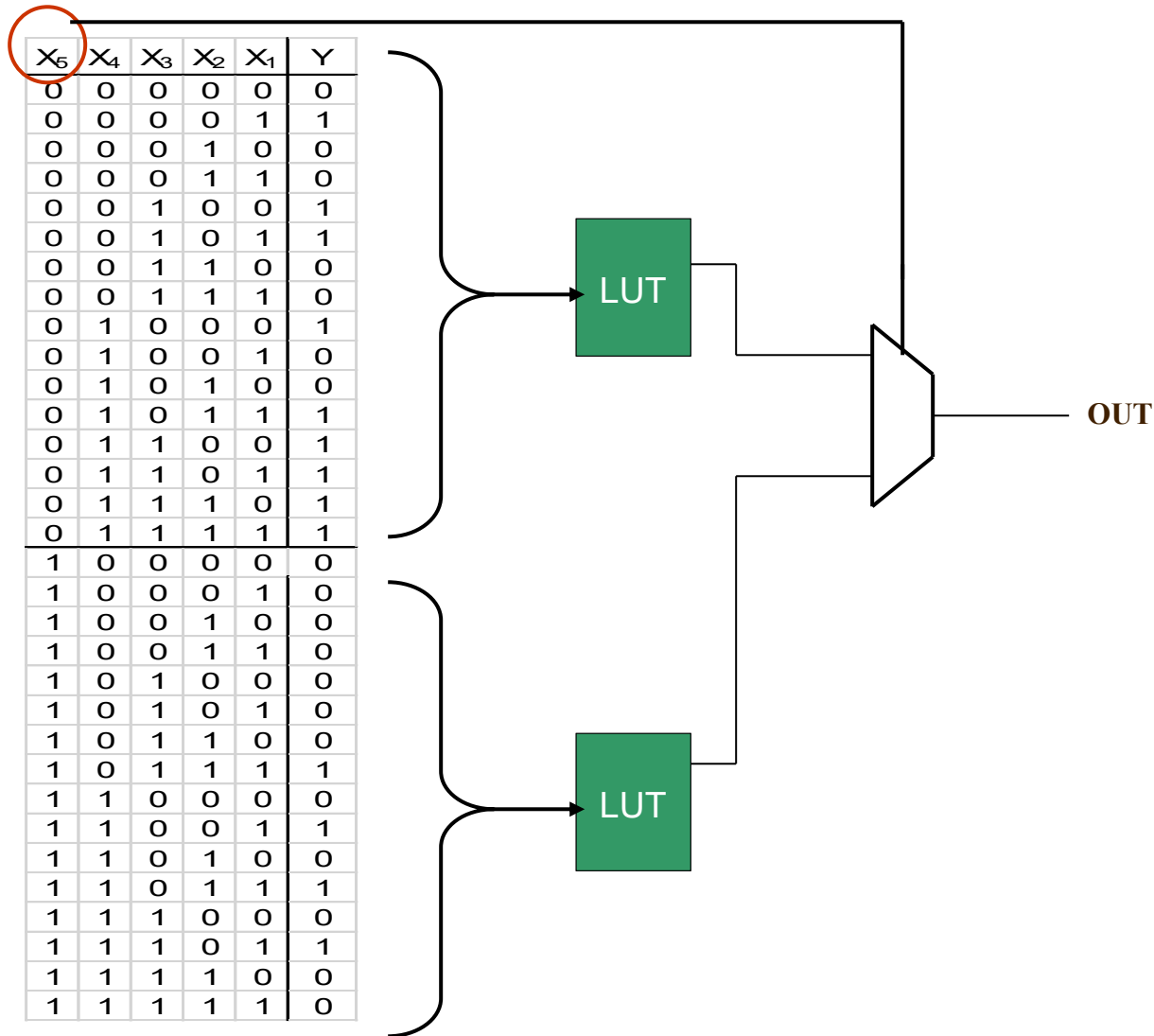


5-Input Functions implemented using two LUTs

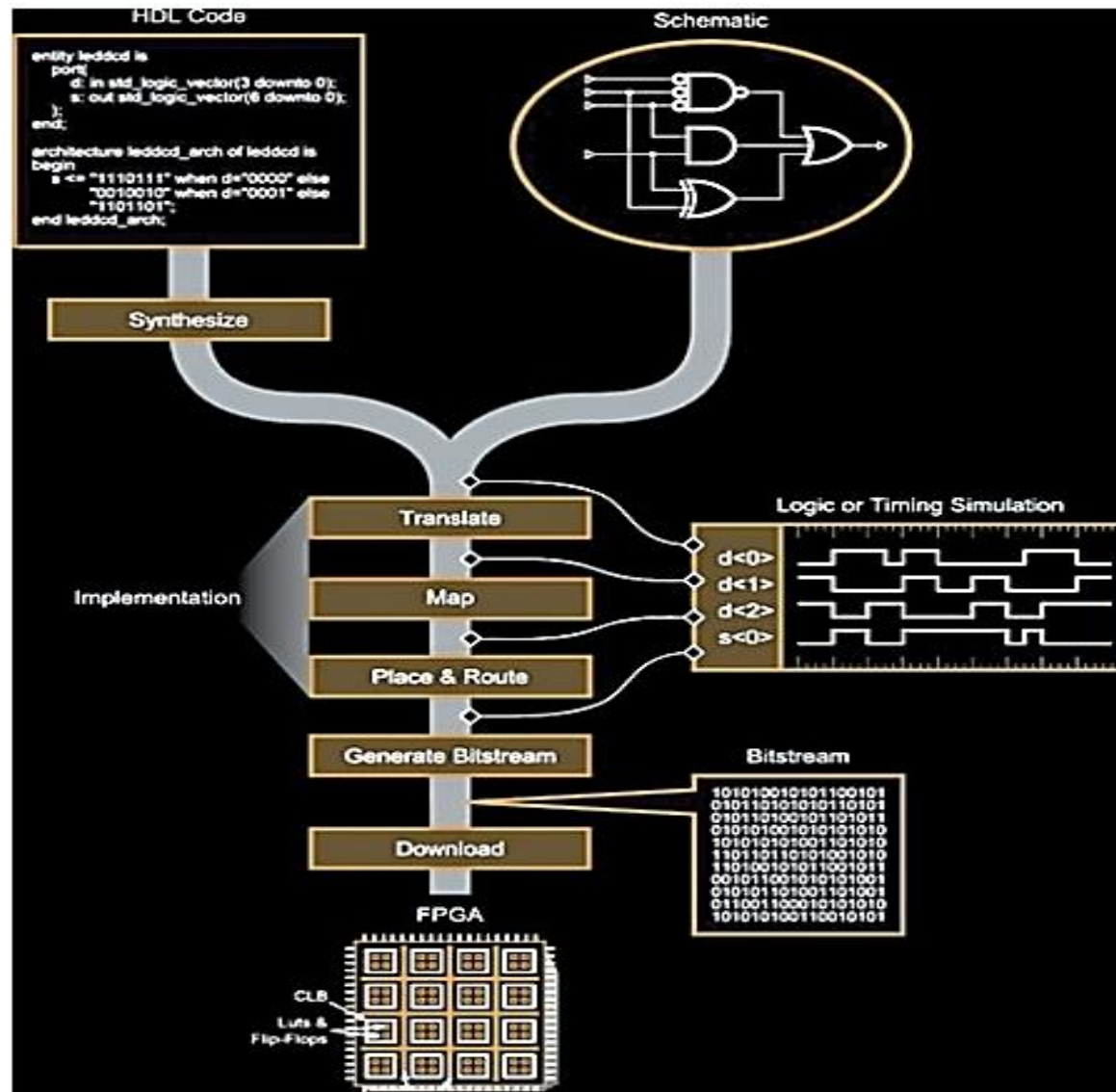
- One CLB Slice can implement any function of 5 inputs
- Logic function is partitioned between two LUTs
- F5 multiplexer selects LUT



5-Input Functions implemented using two LUTs



FPGA Design Flow



FPGA Design Flow

1. Design Entry

- There are different techniques for design entry. Schematic based, Hardware Description Language and combination of both etc.. Selection of a method depends on the design and designer.
- The description of the logic can be made using either a schematic editor, a finite state machine (FSM) editor, or a hardware description language (HDL).
- This is done by selecting components from a given library and providing a direct mapping of the design functions to selected computing blocks.
- When designs with a very large amount of function become difficult to manage graphically, HDL may be used to capture the design either in a structural or in a behavioral way.
- Besides VHDL and Verilog, which are the most established HDLs, several C-like languages are also available such as Handel-C, Impulse C, and SystemC

FPGA Design Flow

2. Synthesis

- This process translates the VHDL / Verilog code into a device netlist format for depicting a complete circuit with logical elements (gates, flip flops, etc...) .
- Synthesis involves checking the code syntax and analyzing the hierarchy of the design architecture.
- Next, the code is compiled along with optimization and the generated netlist is saved as a .ngc file.

3. Implementation

This process consists a sequence of three steps

1. Translate
2. Map
3. Place and Route

FPGA Design Flow

- **Translate** process combines all the input netlists and constraints to a logic design file. Here, defining constraints is nothing but, assigning the ports in the design to the physical elements (ex. pins, switches, buttons etc) of the targeted device and specifying time requirements of the design.
- **Map:** process divides the whole circuit with logical elements into sub blocks such that they can be fit into the FPGA logic blocks.
- **The place and route** process places the sub blocks from the map process into logic blocks according to the constraints and connects the logic blocks. Ex. if a sub block is placed in a logic block which is very near to IO pin, then it may save the time but it may effect some other constraint. So trade off between all the constraints is taken account by the place and route process.

4. Device Programming

- The above-mentioned routed design must be loaded and converted into a format supported by the FPGA. Hence, the routed .ngc file is given to the BitGen program, which generates a bitstream file that contains all the programming information for an FPGA.

5. Design Verification

- This is done all along with the design flow for ensuring that the logic behaves as intended. The following simulations are involved in this process:
 - Behavioral Simulation (RTL Simulation)
 - Functional Simulation
 - Static Timing Simulation
- These simulations are done in order to emulate the behavior of the components by providing test patterns to the inputs of the design and observing the outputs.

What does a logic cell do?

- The logic cell architecture varies between different device families.
- Each logic cell combines a few binary inputs (typically between 3 and 10) to one or two outputs according to a Boolean logic function specified in the user program .
- In most families, the user also has the option of registering the combinatorial output of the cell, so that clocked logic can be easily implemented.
- Cell's combinatorial logic may be physically implemented as a small look-up table memory (LUT) or as a set of multiplexers and gates.
- LUT devices tend to be a bit more flexible and provide more inputs per cell than multiplexer cells at the expense of propagation delay.

What does 'Field Programmable' mean?

- Field Programmable means that the FPGA's function is defined by a user's program rather than by the manufacturer of the device.
- A typical integrated circuit performs a particular function defined at the time of manufacture. In contrast, the FPGA's function is defined by a program written by someone other than the device manufacturer.
- Depending on the particular device, the program is either 'burned' in permanently or semi-permanently as part of a board assembly process, or is loaded from an external memory each time the device is powered up.
- This user programmability gives the user access to complex integrated designs without the high engineering costs associated with application specific integrated circuits.

How are FPGA programs created?

- Individually defining the many switch connections and cell logic functions would be a daunting task.
- This task is handled by special software. The software translates a user's schematic diagrams or textual hardware description language code then places and routes the translated design.
- Most of the software packages have hooks to allow the user to influence implementation, placement and routing to obtain better performance and utilization of the device.
- Libraries of more complex function macros (eg. adders) further simplify the design process by providing common circuits that are already optimized for speed or area.

Difference between CPLD and FPGA

Sr. No.	CPLD	FPGA
1.	Complex Programmable Logic Device	Field Programmable Gate Array
2.	It is collection PLDs.	It is collection of CLBs.
2.	Gate density up to 10000 gates	1000-few hundred 1000s gates
4.	Interconnection wise it is complex so called as CPLD.	Programmable at field or site so called as FPGA.
5.	PAL like blocks used as PLDs.	CLBs used as building blocks.
6.	AND-OR arrays are there in PAL like blocks.	LUTs are there in CLBs.
7.	Configuration context is stored in ROM.	Configuration context is stored in RAM.
8.	Configuration context is Non-volatile.	Configuration context is volatile.